



Solar ASIC
AUTOMATISATION

Guide Installation Complete Guide

Automate your ASIC miners on solar energy.
Zero daytime grid usage | Solar mining | Bear market proof.

Solar

Crypto

Home Assistant

Automation V1.0.37



VERSION

1.0.37

AUTHOR

Halo44

LICENSE

Free & Open

HARDWARE

Refoss + HA



Contents

01	The Concept Why Solar ASIC? Bear market + solar
02	Architecture Overview of the complete system
03	Hardware & Wiring Refoss EM06P, smart plugs, CT placement
04	Home Assistant Integration and base configuration
05	configuration.yaml Sensors, templates, priority ASICs, revenue
06	Automation V1.0.37 Multi-priority, anti yo-yo, continuous SHED
07	Daily report Reliable 8PM notification (history_stats)
08	HTML Dashboard Live interface, kWh charts, i18n
09	Mining pools 7 pools: F2Pool, K1Pool, 2Miners, ViaBTC...
10	Remote access Tailscale VPN - 4G / WiFi access
11	Troubleshooting FAQ and problem solving

01 The Concept

Why Solar ASIC?

In a crypto bear market, mining with grid power at 0.20 EUR/kWh is often a loss. The solution: use only the surplus solar energy from your installation. As long as there is solar surplus, the ASICs run for free. As soon as the surplus drops, they switch off automatically so they never draw from the grid.

The automatic daily cycle

Moment	Behavior
Morning	Sunrise -> growing surplus. Small ASICs start one by one (5s cascade).
Daytime	Maximum surplus. The automation switches to the big priority ASICs (S19, S21...). 100% solar mining, zero grid.
Evening	Surplus drops. ASICs turn off progressively (reverse order, LIFO).
Night	All ASICs off. No grid consumption.

Key difference vs v1: the V1.0.37 automation manages a **list of priority ASICs**. In the morning, small ASICs fill the surplus; as soon as there is enough sun for a big miner, the small ones are cut while it starts, then return in the remaining surplus. Automatic cascade for 1, 2, 3 big ASICs or more.

Three pillars

- **Solar first:** PV production is measured in real time via 6 CT clamps.
- **Smart decision:** Home Assistant computes the surplus and controls each plug.
- **Anti yo-yo:** delays and cooldowns avoid switching on/off due to passing clouds.

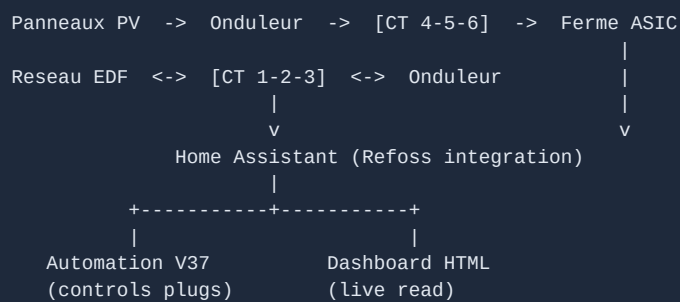
02 Architecture

System overview

Five links connect your panels to your ASICs. Each has a precise role in the measuring and decision chain.

Link	Role
1. PV panels	Produce the energy (e.g. 12 kWp)
2. Refoss EM06P	6 CT clamps measure grid (CT 1-2-3) and ASIC farm (CT 4-5-6)
3. Home Assistant	Brain: computes the surplus, decides which ASICs to run (Raspberry Pi)
4. Smart plugs	Execute: WiFi / Zigbee / Matter (Tuya, Shelly...)
5. ASIC miners	Mine: IceRiver, Goldshell, Antminer...

Data flow



Strength	Detail
Automatic 24/7	The automation runs continuously, cycle every 5 seconds
Anti yo-yo	30s cooldown + 10 min priority boot
Remote access	Tailscale VPN to check on 4G
Grid cost	0 EUR to mine during the day

03 Hardware & Wiring

Wiring diagram - Refoss CT clamps

The Refoss EM06P has 6 measurement channels (6 current clamps). Their placement is crucial for the surplus calculations to be correct.

Clamps	Placement	Measurement
CT 1, 2, 3	Grid cables, between the meter and the inverter	Grid import/export (sign: + import, - export)
CT 4, 5, 6	Inverter output cables (AC side)	ASIC farm consumption (always positive)

IMPORTANT: the Refoss voltage cable (L1 / L2 / L3 / N) must be wired on the same circuit as CT 1-2-3, otherwise the power measurements will be wrong.

Hardware list

Item	Recommended model	Use
6-channel CT meter	Refoss EM06P (or 2x Shelly 3EM)	Measure solar + grid
Smart plugs	Tuya / Shelly / Matter, 1 per ASIC	Controlled on/off
Mini-PC / Pi	Raspberry Pi 3B+ minimum or PC	Hosts Home Assistant
ASIC miners	IceRiver, Goldshell, Antminer...	Your choice by budget

Plug tip: check the max amperage of each plug. An Antminer S19 (~3400W) needs a dedicated 16A plug. Small IceRivers (~100W) work on standard 10A plugs.

04 Home Assistant

Base configuration

Before coding the sensors, integrate the hardware and prepare the HA environment.

Preliminary steps

- 1. Refoss integration:** Settings -> Devices & services -> Add -> Refoss. The 6 sensor.em06_channel_X_power sensors appear.
- 2. Plug integration:** add your plugs (Tuya, Shelly...). Note their entity_id (Dev tools -> States -> filter 'switch.').
- 3. Long-lived token:** Profile -> Security -> Create token. Copy it for the dashboard.
- 4. www folder:** create /config/www/ if it does not exist (to host the dashboard).

CORS: for the HTML dashboard to access the HA API, add the http: block with cors_allowed_origins in configuration.yaml (see section 05).

Recorder (history)

The dashboard shows day/week/month charts. For this, HA must keep the sensor history for at least 31 days:

```
recorder:
  purge_keep_days: 31
  commit_interval: 60
  include:
    entities:
      - sensor.production_solaire
      - sensor.consommmation_ferme
      - sensor.consommmation_reseau
      - sensor.injection_reseau
      - sensor.asic_allumes_count
```

05 configuration.yaml

Computed sensors (Jinja2 templates)

These sensors turn the raw CT measurements into usable values: solar production, surplus, grid import/export, ASIC count.

What you must adapt

- **sensor.em06_channel_X_power** -> the real names of your Refoss sensors
- **switch.asic_small_1 ...** -> the entity_id of your plugs
- **100, 360, 3400** -> the real consumption (W) of each ASIC

```
template:
- sensor:
  - name: "reseau_total"          # CT 1-2-3 (+ import / - export)
    unit_of_measurement: "W"
    state: >
      {% set p1 = states('sensor.em06_channel_1_power')|float(0) %}
      {% set p2 = states('sensor.em06_channel_2_power')|float(0) %}
      {% set p3 = states('sensor.em06_channel_3_power')|float(0) %}
      {{ (p1 + p2 + p3) | round(1) }}

  - name: "injection_reseau"      # surplus to grid
    state: >
      {% set p = states('sensor.reseau_total')|float(0) %}
      {{ (p * -1) if p < 0 else 0 }}

  - name: "consommation_ferme"    # CT 4-5-6 (ASIC load)
    state: >
      {% set p4 = states('sensor.em06_channel_4_power')|float(0) %}
      {% set p5 = states('sensor.em06_channel_5_power')|float(0) %}
      {% set p6 = states('sensor.em06_channel_6_power')|float(0) %}
      {{ [p4 + p5 + p6, 0] | max | round(1) }}

  - name: "production_solaire"    # farm + export
    state: >
      {% set f = states('sensor.consommation_ferme')|float(0) %}
      {% set i = states('sensor.injection_reseau')|float(0) %}
      {{ (f + i) | round(1) }}
```

Priority ASICs (source of truth)

The heart of the V1.0.37 system: a single list defines which ASICs are priority (the big miners). The automation reads this list and handles everything automatically. To add an ASIC (e.g. a new S21), just add a line here - no automation change needed.

```
- name: "asic_prio_config"
  state: "ok"
  attributes:
    asics: >-
      {{ [
        {'entity': 'switch.asic_big_1', 'power': 3400, 'name': 'S19'}
      ] }}
  # Multiple priorities:
  # {'entity': 'switch.s19', 'power': 3450, 'name': 'S19'},
  # {'entity': 'switch.s21_1', 'power': 3500, 'name': 'S21 #1'},
  # {'entity': 'switch.s21_2', 'power': 3500, 'name': 'S21 #2'}
```

Five derived sensors (copy as-is) automatically compute: the power needed for the next start, the next ASIC to turn on, the last one on, and the count of active priority ASICs.

Sensor	Role
<code>asic_prio_power_needed</code>	Total power required to start the next priority
<code>asic_prio_next_entity</code>	entity_id of the next ASIC to turn on (FIFO)
<code>asic_prio_last_entity</code>	entity_id of the last one on (first to cut, LIFO)
<code>asic_prio_count_on</code>	Number of priority ASICs currently ON
<code>binary_sensor.asic_prio_add_ready</code>	ON when solar production covers the next priority

Mining revenue in euros

Two template sensors multiply each pool's production (in crypto) by the price in euros fetched via CoinGecko, to display real revenue in EUR.

```
- platform: rest          # crypto prices, 1 call / 10 min
  name: "crypto_prices_eur"
  resource: "https://api.coingecko.com/api/v3/simple/price
    ?ids=bitcoin,kaspa,alephium&vs_currencies=eur"
  scan_interval: 600
  json_attributes: [bitcoin, kaspa, alephium]
```


06

Automation V1.0.37

Multi-priority logic

The V37 automation runs in single mode (one instance at a time) with 5 triggers. It contains no hard-coded ASIC name: everything comes from sensor.asic_prio_config.

Trigger	Condition	Action
cycle_normal	Every 5s	Continuous regulation: adds/removes small ASICs
maybe_prio_add	Prod >= need, 1 min	Starts the next priority ASIC
sustained_import	Grid > 250W, 5 min	Reduces small ASICs
production_drop	Prod < 2800W, 5 min	Cuts the last priority (evening)
endofday_cleanup	Grid > 50W, 10 min	Cuts all remaining small ASICs (night)

Priority start sequence

When solar production covers the next big ASIC's power for 1 minute:

- All small ASICs are cut (free the 230V bus)
- After 25s, the big ASIC starts
- For 10 min, adding small ASICs is **blocked** (the big one ramps up ~5 min)
- Once stabilized, small ASICs return in the remaining surplus

Why the 10 min block? A big ASIC (S19) does not draw its 3400W instantly: it takes ~5 min to ramp up. Without the block, the ADD cycle would see a false surplus, turn on small ASICs, then have to cut them when the big one reaches full power -> grid import and yo-yo. The prio_boot_ok variable prevents this.

Continuous SHED - the key fix

The sustained_import trigger fires only once. To react to a persistent import, cycle_normal includes a SHED branch: if grid > 50W and small ASICs remain, it cuts one every 5s (with 30s cooldown). This brings consumption below production progressively, without yo-yo.

```
# In cycle_normal:
- choose:
  - conditions:
    - "{{ grid_total > 50 and nb_small_on > 0
      and switch_cooldown_ok }}"
  sequence:
    - action: switch.turn_off
      target: {entity_id: "{{ small_on[-1] }}"}
```

07 Daily report

Reliable report on the Home Assistant side

Every day at 8PM, an HA automation sends a full report via Telegram, ntfy and the HA Companion app. Unlike a browser version, it runs 24/7: mining durations are therefore always exact, even if the dashboard is closed.

history_stats sensors

Home Assistant continuously computes the ON time of each ASIC (reset at midnight). These sensors feed the report with reliable durations.

```
sensor:
  - platform: history_stats
    name: "asic_big_1 heures_jour"
    entity_id: switch.asic_big_1
    state: "on"
    type: time
    start: "{{ today_at('00:00') }}"
    end: "{{ now() }}"
```

Report content

Block	Detail
Energy	Solar produced, ASIC load, grid import/export (kWh)
Durations	Total mining + hours per ASIC, grouped by model (history_stats)
Revenue	Previous day (D-1) mining revenue in euros
Net	Revenue - grid cost (export not counted without a buyback contract)

Net without export: without an EDF buyback contract, exported energy is not paid. Net = mining revenue - grid import cost. Export is still shown for information.

Notification channels

- **Telegram:** via a bot (token + chat_id), instant message
- **ntfy:** open-source push, custom topic (ntfy.sh app)
- **HA Companion:** native notification on the Home Assistant mobile app

08 HTML Dashboard

Live visual interface

A single HTML file (HTML + CSS + JS) to drop into /config/www/. It reads the HA API and displays production, consumption, animated energy flow, mining revenue and history.

#	Step	Detail
1	Copy the file	/config/www/dashboard_mining.html
2	Copy secrets.example.js	-> secrets.js (your tokens and IDs)
3	Create an API token	HA Profile -> Security -> Long-lived tokens
4	Open the dashboard	http://HA_IP:8123/local/dashboard_mining.html

Variables to customize (secrets.js)

Variable	Default	Replace with
HA_URL_LOCAL	http://192.168.1.XX:8123	Local IP of your HA
HA_URL_VPN	http://100.X.X.X:8123	Tailscale IP (optional)
HA_TOKEN	eyJhbGciOi...	Your HA API token
MINING_COINS	[...]	List of your mined cryptos
F2POOL_TOKEN	your_token	Pool API key

Dashboard features

- **Animated energy flow:** solar -> farm -> ASIC, with grid import/export
- **Day chart:** power curves (W) over 24h
- **Week/month chart:** daily kWh bars (Solarman style)
- **Mining revenue:** per crypto, real time and in euros
- **Yesterday report:** revenue - grid cost of the previous day
- **Multi-language:** French, English, German, Spanish
- **Next start:** countdown and threshold of the next ASIC

Bar charts: for the week and month views, the dashboard integrates power (W) into energy (kWh) per day using the trapezoidal method, then displays 4 bars per day (solar, farm, import, export) - just like the Solarman app.

09 Mining pools

7 supported pools

Solar ASIC can display revenue from any pool. The architecture is universal: Home Assistant fetches data server-side (no CORS issue), the dashboard reads the HA sensors. All sensors must be named `sensor.f2pool_SYMBOL_day` and `sensor.f2pool_SYMBOL_unpaid` (the `f2pool_` prefix is conventional, regardless of the actual pool).

Pool	Cryptos	Auth	Type	Difficulty
F2Pool	BTC, ALPH, KAS, LTC...	Token V2 (POST)	command_line	Hard
K1Pool	KAS, RXD, ZIL...	Wallet (GET)	command_line	Easy
2Miners	ETC, RVN, KAS, ZEC...	Wallet (GET)	rest:	Easy
ViaBTC	BTC, ETH, LTC...	API Key (header)	rest:	Medium
NiceHash	BTC (all algos)	HMAC complex	command_line	Hard
HeroMiners	KAS, ALPH, RXD...	Wallet (GET)	rest:	Easy
Kryptex	BTC, KAS, RXD...	Token (GET)	rest:	Medium

Universal method (any new pool)

1. Test the API from PowerShell (Invoke-RestMethod) and note the JSON structure
2. Create an HA sensor: rest: if simple GET, command_line: if POST or complex auth
3. Create template sensors `f2pool_SYMBOL_day` and `f2pool_SYMBOL_unpaid`
4. Add the crypto to `MINING_COINS` in `secrets.js`
5. Restart HA and check in Dev tools -> States

2Miners example (public GET, wallet)

Public API without authentication: the wallet address is enough. Subdomain per crypto (etc.2miners.com, kas.2miners.com...). Returns balance, 24h reward and workers.

```
rest:
- resource: "https://etc.2miners.com/api/accounts/YOUR_WALLET"
  scan_interval: 300
  sensor:
    - name: "2miners_etc_raw"
      value_template: >
        {{ (value_json.stats.balance|float(0) / 1e9) | round(6) }}
      json_attributes: [stats, "24hreward", workersOnline]

template:
- sensor:
    - name: "f2pool_etc_day"      # generic dashboard name
      state: >
        {% set r = state_attr('sensor.2miners_etc_raw', '24hreward') %}
        {{ (r|float(0) / 1e9) | round(6) if r else 0 }}
```

Divisor per coin: ETC = / 1,000,000,000, KAS = / 100,000,000, ZEC = / 100,000,000. Always check the raw response before choosing the divisor.

ViaBTC example (GET + API Key header)

API key in the header (no HMAC signature). Separate endpoints for hashrate and profits.

```
rest:
- resource: "https://www.viabtc.com/res/openapi/v1/profit?coin=BTC"
  scan_interval: 300
  headers:
    Authorization: "YOUR_API_KEY_VIABTC"
  sensor:
    - name: "viabtc_btc_profit"
      value_template: "{{ value_json.data.total_profit|float(0) }}"
```

NiceHash example (HMAC - Python script)

NiceHash uses complex HMAC authentication: each request is signed with timestamp + nonce. Not possible directly with HA sensors -> use a Python script on the Pi, called via `command_line`.

```
# /config/scripts/nicehash_balance.py
import hashlib, hmac, uuid, time, requests
API_KEY='...'; API_SECRET='...'; ORG_ID='...'
def sign(method, path, ts, nonce, body=''):
    msg = f'{API_KEY}\x00{ts}\x00{nonce}\x00\x00{ORG_ID}'
    msg += f'\x00\x00{method}\x00{path}\x00{body}'
    return hmac.new(API_SECRET.encode(), msg.encode(),
                    hashlib.sha256).hexdigest()
ts=str(int(time.time()*1000)); nonce=str(uuid.uuid4())
path='/main/api/v2/accounting/account2/BTC'
sig=sign('GET', path, ts, nonce)
headers={'X-Time':ts, 'X-Nonce':nonce, 'X-Organization-Id':ORG_ID,
        'X-Auth':f'{API_KEY}:{sig}'}
r=requests.get(f'https://api2.nicehash.com{path}', headers=headers)
print(r.json().get('totalBalance',{}).get('available','0'))
```

```
# configuration.yaml
command_line:
  - sensor:
      name: "nicehash_btc_balance"
      scan_interval: 300
      command: "python3 /config/scripts/nicehash_balance.py"
      value_template: "{{ value | float(0) }}"
      command_timeout: 20
```

Other pools endpoints

Pool	Endpoint	Key fields
K1Pool	GET /api/miner/{coin}/{wallet}	miner.pendingBalance, miner.coinsPerDay
HeroMiners	GET /api/stats_address?address={w}	stats.balance, stats.paid_today
Kryptex	GET /api/v1/hashrates?token={t}	data.balance, data.todayIncome
F2Pool	POST /v2/assets/balance	balance, yesterday_income

10 Remote access

Tailscale VPN

Tailscale creates a private network between your devices. You access your dashboard from anywhere (4G, external WiFi) as if you were home, without opening any port.

#	Step	Detail
1	HA Add-on	Settings -> Add-ons -> Tailscale -> Start
2	Tailscale account	tailscale.com (free) -> note the 100.x.x.x IP
3	Mobile app	Install Tailscale iOS/Android, same account
4	Edit secrets.js	Set HA_URL_VPN with your 100.x.x.x IP

```
const HA_URL_LOCAL = 'http://192.168.1.XX:8123';
const HA_URL_VPN   = 'http://100.X.X.X:8123';
// Auto-switch based on detected network
let HA_URL = location.hostname.startsWith('192.168')
  ? HA_URL_LOCAL : HA_URL_VPN;
```

Auto-switch: the dashboard detects whether it is opened locally (192.168.x.x) or remotely, and picks the right URL. Transparent for the user.

11

Troubleshooting

FAQ and problem solving

Problem	Solution
Dashboard shows ERROR	Invalid token or wrong IP. Recreate the token in HA and check the URL in secrets.js.
Sensors unavailable	Reload templates: Dev tools -> YAML -> Template entities.
ASICs do not turn on	Check that the V37 automation is active and surplus exceeds the threshold for the required duration.
Plug IDs changed	Dev tools -> States -> filter 'switch.' to find the new entity_id.
Day chart empty	Normal for the first 24h. History accumulates progressively.
Week/month chart empty	Check recorder: purge_keep_days >= 31 and that sensors are in include:.
Grid import despite surplus	The filter ignores import < 20W (Refoss CT noise). Beyond that, check the clamp calibration.
Crypto prices unknown	Wait 10 min (1st CoinGecko call) and check the Pi's internet access.
8PM report not received	Check Telegram/ntfy credentials and that the report automation is active. Test it manually.
ASIC yo-yo	Increase the cooldown (30s) or the priority boot delay (10 min) in the automation.
Not reachable on 4G	Tailscale must be active on the phone AND on HA. Check the 100.x.x.x IP in the app.

12 Successful install!

This project is shared for free. Improve it, distribute it, share it with the community.

Final checklist

- [V] Refoss EM06P integrated in HA (6 power sensors visible)
- [V] configuration.yaml with templates, priority ASICs and revenue
- [V] V1.0.37 automation active (multi-priority, SHED, anti yo-yo)
- [V] Daily 8PM report automation configured (Telegram/ntfy/HA)
- [V] HTML dashboard customized in /config/www/
- [V] secrets.js filled (never committed to GitHub)
- [V] Tailscale remote access (optional)

Solar ASIC v4 - Halo44 - Free license.

Let's mine clean. Source code and examples available on GitHub.